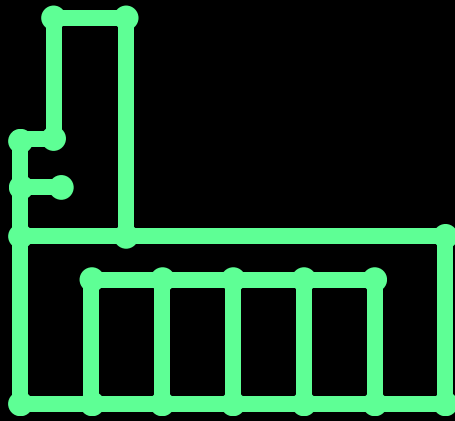
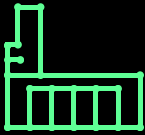




KHARAGPUR
BLOCKCHAIN SOCIETY

SOPHOMORE
SELECTIONS
TASKS 2026



TASK-1

CAMPUS FUND

Crowdfunding with Milestone-Based Release

BACKGROUND

KGP's student clubs raise money on-chain for events and projects. But a single lump-sum payout to the organizer, the moment a goal is hit, is risky - what if the organizer takes the money and the event never happens? Contributors want some assurance that funds are released only as work is actually done, not all at once on trust.

You're building a crowdfunding contract where funds are locked, released in stages tied to milestones the contributors themselves approve, and refundable if the campaign fails or contributors lose confidence partway through.

FUNCTIONAL REQUIREMENTS

A. Campaign Creation :

- Any address can create a campaign with: a funding goal, a deadline, a description, and a list of 2-4 milestones, each with a percentage of total funds to release.
- Milestone percentages for a campaign must sum to 100%.

B. Contribution

- Anyone can contribute ETH to an open (pre-deadline) campaign.
- The contract must track each contributor's amount per campaign (for refund and for milestone-approval voting weight).

C. Milestone Approval (Contributor Governance)

- Once the goal is met, the organizer can request release of the next pending milestone.
- Contributors vote (weighted by their contribution amount) to approve or reject releasing that milestone's funds. If a majority (by ETH-weight, not headcount) approves, the milestone's portion is released to the organizer.
- A contributor cannot vote twice on the same milestone request.

D. Refunds

- If the deadline passes without the goal being met, contributors can pull refunds for their full contribution.
- If a milestone vote is rejected, remaining (unreleased) funds should become refundable to contributors proportional to their contribution, and no further milestones should be releasable.

E. View Functions

- Campaign details, per-milestone status (pending/approved/rejected/released), and a contributor's own contribution + refund eligibility.



SECURITY & EDGE CASES TO HANDLE

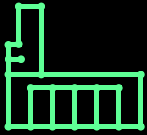
- Reentrancy: All ETH-transferring functions (refunds, milestone releases) must follow checks-effects-interactions and be safe against reentrancy.
- No Double Refund / Double Release: A contributor can't refund the same contribution twice; a milestone can't be released twice.
- Percentage Integrity: Milestone percentages must be validated at creation time (sum to exactly 100%) - reject campaigns that don't.
- Rounding: Handle integer division carefully when splitting a milestone percentage of a total (don't silently lose or duplicate wei).
- Dead Campaigns: A campaign with zero contributors reaching its deadline shouldn't break any view function.
- Access Control: Only the organizer can request a milestone release; only actual contributors can vote or claim refunds.

DELIVERABLES

- Solidity contract in Remix (single contract or small set).
- Simulated run: a campaign that hits its goal, has one milestone approved and released, one milestone rejected (triggering partial refunds), from at least 3 different contributor addresses.
- Code on GitHub + screenshots of each step and resulting balances.
- Half a page explaining your reentrancy protection and how you handled percentage rounding.

BONUS - OPTIONAL

- Let contributors change their vote before a milestone request is finalized.
- Add a minimum contributor turnout requirement for a milestone vote to count.
- Emit events for every state-changing action (creation, contribution, milestone requested/approved/rejected/released, refund claimed).



TASK-2

CERTIFY ME

Multi-Issuer Certificate Registry with Revocation History

BACKGROUND

A club wants a tamper-proof, publicly verifiable way to say "this address holds a valid certificate for X", usable by multiple authorized issuers (not just one admin), auditable (nothing silently disappears), and resistant to an issuer going rogue.

FUNCTIONAL REQUIREMENTS

A. Issuer Management

- The contract owner can add or remove addresses as authorized issuers.
- Only authorized issuers (not just the owner) can issue certificates.

B. Certificate Issuance

- An issuer can issue a certificate to a student's address for a specific workshop/course, with a title, date, and the issuing address recorded.
- The same student cannot receive a duplicate certificate for the same workshop from any issuer.

C. Revocation with Audit Trail

- Only the original issuing address, or the owner, can revoke a certificate, with a required reason string.
- Revoked certificates must remain queryable - including who issued it, who revoked it, and why - never deleted outright.
- A revoked certificate must clearly fail a "is this valid?" check while still being visible in a "show me everything ever issued to this address" history check.

D. Verification

- Anyone can check whether an address holds a currently valid certificate for a given workshop.
- Anyone can retrieve the full certificate history (valid + revoked) for a given address.

E. Issuer Accountability

- Anyone can query all certificates ever issued by a specific issuer address (useful if an issuer is later suspected of abuse).



SECURITY & EDGE CASES TO HANDLE

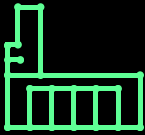
- Access Control Layering: Owner vs. issuer vs. public should each have clearly different permissions - a removed issuer must immediately lose the ability to issue or revoke, even for certificates they issued in the past.
- No Silent Data Loss: Revocation must be an additive state change, never a deletion.
- Duplicate Prevention Across Issuers: Two different issuers shouldn't be able to independently issue the same student the same certificate twice.
- Owner Self-Removal: Decide and document what happens if the owner tries to remove themselves as an issuer (allowed? blocked? - either is fine, just be intentional and explain your choice).

DELIVERABLES

- Solidity contract in Remix.
- Simulated run: owner adds 2 issuers, both issue certificates (including one attempted duplicate that correctly fails), one issuer revokes a certificate they issued, owner removes an issuer who then correctly fails to issue further certificates.
- Code on GitHub + screenshots.
- Half a page on your access-control design and why revocation doesn't delete data.

BONUS - OPTIONAL

- Let a student "acknowledge" a certificate (a lightweight two-step issuance: issuer proposes, student confirms) so certificates can't be spammed onto addresses that never asked for them.
- Add a certificate expiry date, after which validity checks return false automatically without needing manual revocation.



TASK-3

TASK BOUNTY

Bounty Board with Escrowed Payout and Dispute Fallback

BACKGROUND

Club members want to post paid micro-bounties (design a poster, fix a bug) with the ETH locked upfront, but a plain "poster picks a winner" system has a problem: what if the poster goes silent and never picks anyone, leaving submitters' work unrewarded and funds stuck forever? You need a system with a built-in fallback so funds can never be permanently trapped, while still preventing the poster from just keeping the money without picking a winner.

FUNCTIONAL REQUIREMENTS

A. Post Bounty

- Anyone can post a bounty with a description, a submission deadline, and ETH locked at creation time.

B. Submit Work

- Any address other than the poster can submit an entry (text/link) before the deadline. Multiple entries per bounty are allowed, one entry per address.

C. Winner Selection

- Only the poster can select a winner from the submitters, and only before a set "decision deadline" (some window after the submission deadline). Selecting a winner immediately and atomically pays out the locked ETH.

D. Timeout Fallback (No Stuck Funds)

- If the poster does not select a winner before the decision deadline:
- If there were zero submissions, the poster can reclaim their own locked ETH.
- If there was at least one submission, describe and implement a fair fallback (acceptable options: split the bounty evenly among all submitters, or let submitters themselves vote on a winner). Document which you chose and why.

E. View Functions

- Bounty details, list of submitters, current status (Open/AwaitingDecision/Paid/Expired), and winner (if decided).



SECURITY & EDGE CASES TO HANDLE

- **Guaranteed Exit Path:** Every wei locked into the contract must have a reachable path out - no state should be able to trap funds permanently.
- **Reentrancy:** Payout functions (winner payment, timeout fallback, poster reclaim) must be reentrancy-safe.
- **No Self-Selection:** A poster cannot submit to, or select, themselves as the winner of their own bounty.
- **State Enforcement:** A bounty already paid out cannot be submitted to, re-decided, or paid again.
- **Poster Griefing:** A poster shouldn't be able to withdraw locked funds for themselves once submissions exist, except via the documented timeout fallback rule.

DELIVERABLES

- Solidity contract in Remix.
- Simulated run: one bounty resolved normally (poster picks a winner before the decision deadline), one bounty where the poster goes silent past the decision deadline with submissions present, showing your fallback logic executing correctly, and one bounty with zero submissions correctly refunded to the poster.
- Code on GitHub + screenshots.
- Half a page explaining your timeout fallback design choice and why it guarantees no funds get permanently stuck.

BONUS - OPTIONAL

- Let the poster cancel a bounty before any submissions exist, reclaiming funds immediately.
- Add a small "reputation" counter per address tracking how many bounties they've won, purely as public on-chain data.